

pow (power), função 86
 protótipo de função 86
 regra de aninhamento 101
 regra de empilhamento 100
 repetição definida 80
 repetição indefinida 80

rvalue ('right value') 98
 tabelas verdade 96
 valor final de uma variável de controle 80
 valor inicial de uma variável de controle 80
 variável de controle 80

■ Exercícios de autorrevisão

4.1 Preencha os espaços nas sentenças.

- a) A repetição controlada por contador também é conhecida como repetição _____, pois sabemos com antecedência quantas vezes o loop será executado.
- b) A repetição controlada por sentinela também é conhecida como repetição _____, pois não sabemos com antecedência quantas vezes o loop será executado.
- c) Na repetição controlada por contador, um(a) _____ é usado(a) para contar o número de vezes que um grupo de instruções deve ser repetido.
- d) O comando _____, quando executado em uma estrutura de repetição, faz com que a próxima iteração do loop seja realizada imediatamente.
- e) O comando _____, quando executado em uma estrutura de repetição ou em um switch, causa uma saída imediata da estrutura.
- f) A _____ é usada para testar uma variável ou expressão em particular para cada um dos valores inteiros constantes que ela pode assumir.

4.2 Indique se as seguintes sentenças são *verdadeiras* ou *falsas*. Explique sua resposta no caso de haver sentenças *falsas*.

- a) O caso default é obrigatório na estrutura de seleção switch.
- b) O comando break é obrigatório no caso default de uma estrutura de seleção switch.
- c) A expressão $(x > y \ \&\& \ a < b)$ é verdadeira se $x > y$ for verdadeiro ou $a < b$ for verdadeiro.
- d) Uma expressão contendo o operador `||` é verdadeira se um ou ambos os seus operandos forem verdadeiros.

4.3 Escreva uma instrução ou um conjunto de instruções para realizar cada uma das seguintes tarefas:

- a) Somar os inteiros ímpares entre 1 e 99 usando uma estrutura for. Considere que as variáveis inteiras soma e contador tenham sido declaradas.
- b) Imprima o valor 333,546372 em uma largura de campo de 15 caracteres com precisões de 1, 2, 3, 4 e 5. Alinhe a saída à esquerda. Quais são os cinco valores impressos?

- c) Calcule o valor de 2.5 elevado à potência 3 usando a função pow. Imprima o resultado com uma precisão de 2 em uma largura de campo de 10 posições. Qual é o valor impresso?
- d) Imprima os inteiros de 1 a 20 usando um loop while e a variável contadora x. Considere que a variável x tenha sido declarada, mas não inicializada. Imprima somente cinco inteiros por linha. [Dica: use o cálculo $x \% 5$. Quando o resultado for 0, imprima um caractere de nova linha, senão imprima um caractere de tabulação.]
- e) Repita o Exercício 4.3 (d) usando uma estrutura for.

4.4 Encontre o erro em cada um dos segmentos de código a seguir, e explique como corrigi-lo.

- a)

```
x = 1;
while ( x <= 10 );
    x++;
}
```
- b)

```
for ( y = .1; y != 1.0; y += .1 )
    printf( "%f\n", y );
```
- c)

```
switch ( n ) {
    case 1:
        printf( "0 número é 1\n" );
    case 2:
        printf( "0 número é 2\n" );
        break;
    default:
        printf( "0 número não é nem 1 nem 2\n" );
        break;
}
```
- d) O código a seguir deveria imprimir os valores de 1 a 10.

```
n = 1;
while ( n < 10 )
    printf( "%d ", n++ );
```

■ Respostas dos exercícios de autorrevisão

4.1 a) definida. b) indefinida. c) variável de controle ou contador. d) continue. e) break. f) estrutura de seleção switch.

4.2 a) Falso. O caso default é opcional. Se nenhuma ação default for necessária, então um caso default não é necessário.

b) Falso. A instrução break é usada para sair da estrutura switch. O comando break não é exigido quando o caso default é o último caso.

c) Falso. As duas expressões relacionais deverão ser verdadeiras para que a expressão inteira seja verdadeira quando se usa o operador &&.

d) Verdadeiro.

4.3 a) soma = 0;

```
for ( contador = 1; contador <= 99; contador += 2 ) {
    soma += contador;
}
```

```
b) printf( "%-15.1f\n", 333.546372 ); /* im-
prime 333.5 */
printf( "%-15.2f\n", 333.546372 ); /* im-
prime 333.55 */
printf( "%-15.3f\n", 333.546372 ); /* im-
prime 333.546 */
printf( "%-15.4f\n", 333.546372 ); /* im-
prime 333.5464 */
printf( "%-15.5f\n", 333.546372 ); /* im-
prime 333.54637 */
```

```
c) printf( "%10.2f\n", pow( 2.5, 3 ) ); /*
imprime 15.63 */
```

d) x = 1;

```
while ( x <= 20 ) {
    printf( "%d", x );
    if ( x % 5 == 0 ) {
        printf( "\n" );
    }
    else {
        printf( "\t" );
    }
    x++;
}
```

ou

```
x = 1;
```

```
while ( x <= 20 ) {
    if ( x % 5 == 0 ) {
        printf( "%d\n", x++ );
    }
    else {
        printf( "%d\t", x++ );
    }
}
```

```
}
```

ou

```
x = 0;
```

```
while ( ++x <= 20 ) {
    if ( x % 5 == 0 ) {
        printf( "%d\n", x );
    }
    else {
        printf( "%d\t", x );
    }
}
```

```
e) for ( x = 1; x <= 20; x++ ) {
    printf( "%d", x );
    if ( x % 5 == 0 ) {
        printf( "\n" );
    }
    else {
        printf( "\t" );
    }
}
```

ou

```
for ( x = 1; x <= 20; x++ ) {
    if ( x % 5 == 0 ) {
        printf( "%d\n", x );
    }
    else {
        printf( "%d\t", x );
    }
}
```

- 4.4** a) Erro: o ponto e vírgula após o cabeçalho do `while` causa um loop infinito.

Correção: substitua o ponto e vírgula por `{` ou remova o `;` e o `}`.

- b) Erro: usar um número em ponto flutuante para controlar uma estrutura de repetição `for`.

Correção: use um inteiro e realize o cálculo apropriado para obter os valores que você deseja.

```
for ( y = 1; y != 10; y++ )
    printf( "%f\n", ( float ) y / 10 );
```

- c) Erro: falta o comando `break` nas instruções para o primeiro `case`.

Correção: inclua um comando `break` ao final das instruções para o primeiro `case`. Isso não constitui necessariamente um erro se você quiser que a instrução do `case 2`: seja executada toda vez que a instrução do `case 1`: for executada.

- d) Erro: uso do operador relacional impróprio na condição de continuação de repetição do `while`.

Correção: use `<=` no lugar de `<`.

Exercícios

- 4.5** Ache o erro em cada um dos seguintes trechos de códigos. (Nota: pode haver mais de um erro.)

- a)** `For ( x = 100, x >= 1, x++ )`

```
printf( "%d\n", x );
```

- b)** O código a seguir deveria imprimir se determinado inteiro é ímpar ou par:

```
switch ( valor % 2 ) {
    case 0:
        printf( "Inteiro par\n" );
    case 1:
        printf( "Inteiro ímpar\n" );
}
```

- c)** O código a seguir deveria ler um inteiro e um caractere e imprimi-los. Suponha que o usuário digite 100 A.

```
scanf( "%d", &intVal );
charVal = getchar();
printf( "Inteiro: %d\nCaractere: %c\n",
        intVal, charVal );
```

- d)** `for ( x = .000001; x == .0001; x += .000001 ) {`

```
    printf( "%.7f\n", x );
}
```

- e)** O código a seguir deveria exibir os inteiros ímpares de 999 a 1:

```
for ( x = 999; x >= 1; x += 2 ) {
    printf( "%d\n", x );
}
```

- f)** O código a seguir deveria exibir os inteiros pares de 2 a 100:

```
contador = 2;
Do {
    if ( contador % 2 == 0 ) {
        printf( "%d\n", contador );
    }
    contador += 2;
} While ( contador < 100 );
```

- g)** O código a seguir deveria somar os inteiros de 100 a 150 (supondo que `total` seja inicializado com 0):

```
for ( x = 100; x <= 150; x++ ); {
    total += x;
}
```

- 4.6** Indique quais valores da variável de controle `x` são impressos por cada uma das seguintes estruturas `for`:

- a)** `for ( x = 2; x <= 13; x += 2 ) {`
- ```
 printf("%d\n", x);
}
```

- b)** `for ( x = 5; x <= 22; x += 7 ) {`
- ```
    printf( "%d\n", x );
}
```

- c)** `for ( x = 3; x <= 15; x += 3 ) {`
- ```
 printf("%d\n", x);
}
```

- d)** `for ( x = 1; x <= 5; x += 7 ) {`
- ```
    printf( "%d\n", x );
}
```

- e)** `for ( x = 12; x >= 2; x -= 3 ) {`
- ```
 printf("%d\n", x);
}
```

a) 1, 2, 3, 4, 5, 6, 7  
b) 3, 8, 13, 18, 23  
c) 20, 14, 8, 2, -4, -10  
d) 19, 27, 35, 43, 51

```

1 #include <stdio.h>
2
3 /* função main inicia a execução do
 programa */
4 int main(void)
5 {
6 int x;
7 int y;
8 int i;
9 int j;
10
11 /* solicita a entrada do usuário */
12 printf("Digite dois inteiros no
 intervalo 1-20: ");
13 scanf("%d%d", &x, &y); /* lê va-
 lores para x e y */
14
15 for (i = 1; i <= y; i++) { /* conta
 de 1 a y */
16
17 for (j = 1; j <= x; j++) { /* conta
 de 1 a x */
18 printf("@"); /* exibe @ */
19 } /* fim do for interno */
20
21 printf("\n"); /* inicia nova linha
 */
22 } /* fim do for externo */
23
24 return 0; /* indica que o programa foi
 concluído com sucesso */
25 } /* fim da função main */

```

indicando que é preciso calcular a média de todos os valores anteriores a 9999.

**4.16** *Problema de impressão de triângulo.* Escreva um programa que imprima os padrões a seguir separadamente, um abaixo do outro. Use loops for para gerar os padrões. Todos os asteriscos (\*) devem ser impressos por uma única instrução printf na forma printf(“\*”) ; (isso faz com que os asteriscos sejam impressos lado a lado). [Dica: os dois últimos padrões exigem que cada linha comece com um número de espaços apropriado.]

**4.17** *Calculando limites de crédito.* Poupar dinheiro vem se tornando algo cada vez mais difícil de se fazer durante períodos de recessão, de modo que as empresas podem estreitar seus limites de crédito para impedir que suas contas a receber (dinheiro devido a elas) se tornem muito grandes. Em resposta a uma recessão prolongada, uma empresa cortou os limites de crédito de seus clientes pela metade. Assim, se um cliente em particular tinha um limite de crédito de R\$ 2.000,00, agora ele é de R\$ 1.000,00. Se um cliente tinha um limite de R\$ 5.000,00, agora ele é de R\$ 2.500,00. Escreva um programa que analise o status de crédito de três clientes dessa empresa. Você receberá as seguintes informações:

- a) O número de conta do cliente.
- b) O limite de crédito do cliente antes da recessão.
- c) O saldo atual do cliente (ou seja, o valor que o cliente deve à empresa).

Seu programa deve calcular e imprimir o novo limite de crédito para cada cliente e deve determinar (e imprimir) quais clientes têm saldo atual superior a seus novos limites de crédito.

**4.18 Programa de exibição de gráfico de barras.** Uma aplicação interessante dos computadores é a de desenhar gráficos e gráficos de barras (às vezes, chamados 'histogramas'). Escreva um programa que leia cinco números (entre 1 e 30). Para cada número lido, seu programa deverá exibir uma linha contendo esse número de asteriscos adjacentes. Por exemplo, se seu programa ler o número sete, ele deverá exibir `*****`.

**4.19 Calculando vendas.** Um varejista on-line vende cinco produtos diferentes cujos preços de revenda aparecem na tabela a seguir:

| Número do produto | Preço de revenda |
|-------------------|------------------|
| 1                 | R\$ 2,98         |
| 2                 | R\$ 4,50         |
| 3                 | R\$ 9,98         |
| 4                 | R\$ 4,49         |
| 5                 | R\$ 6,87         |

Escreva um programa que leia uma série de pares de números da seguinte forma:

- a) Número do produto.
- b) Quantidade vendida durante um dia.

Seu programa deverá usar uma estrutura `switch` para ajudar a determinar o preço de revenda para cada produto. O programa deverá calcular e exibir o valor de revenda total de todos os produtos vendidos na semana anterior.

**4.20 Tabelas verdade.** Complete as tabelas verdade a seguir preenchendo cada espaço com 0 ou 1.

| Condição1 | Condição2 | Condição1 && Condição2 |
|-----------|-----------|------------------------|
| 0         | 0         | 0                      |
| 0         | não zero  | 0                      |
| não zero  | 0         | _____                  |
| não zero  | não zero  | _____                  |

| Condição1 | Condição2 | Condição1    Condição2 |
|-----------|-----------|------------------------|
| 0         | 0         | 0                      |
| 0         | não zero  | 1                      |
| não zero  | 0         | _____                  |
| não zero  | não zero  | _____                  |

| Condição1 | !Condição1 |
|-----------|------------|
| 0         | 1          |
| não zero  | _____      |

**4.21** Reescreva o programa da Figura 4.2 de modo que a inicialização da variável contador seja feita na declaração, e não na estrutura `for`.

**4.22 Nota média.** Modifique o programa da Figura 4.7 de modo que seja possível calcular a nota média da classe.

**4.23 Calculando juros compostos com inteiros.** Modifique o programa da Figura 4.6 de modo que ele use apenas inteiros para calcular os juros compostos. [Dica: trate todos os valores monetários como números inteiros de centavos. Depois 'quebre' o resultado em sua parte de real e em sua parte de centavos usando as operações de divisão e módulo, respectivamente. Insira uma vírgula.]

**4.24** Considere `i = 1`, `j = 2`, `k = 3` e `m = 2`. O que cada uma das seguintes instruções imprime?

- a) `printf( "%d", i == 1 );`
- b) `printf( "%d", j == 3 );`
- c) `printf( "%d", i >= 1 && j < 4 );`
- d) `printf( "%d", m <= 99 && k < m );`
- e) `printf( "%d", j >= i || k == m );`
- f) `printf( "%d", k + m < j || 3 - j >= k );`
- g) `printf( "%d", !m );`
- h) `printf( "%d", !(j - m) );`
- i) `printf( "%d", !(k > m) );`
- j) `printf( "%d", !(j > k) );`

**4.25 Tabela de equivalência decimal, binária, octal e hexadecimal.** Escreva um programa que imprima uma tabela dos equivalentes binário, octal e hexadecimal dos números decimais no intervalo de 1 a 256. Se você não estiver acostumado com esses sistemas numéricos, leia o Apêndice C antes de tentar realizar esse exercício.

**4.26 Calculando o valor de  $\pi$ .** Calcule o valor de  $\pi$  a partir da série infinita

$$\pi = 4 - \frac{4}{3} + \frac{4}{5} - \frac{4}{7} + \frac{4}{9} - \frac{4}{11} + \dots$$

Imprima uma tabela que mostre o valor de  $\pi$  aproximado por um termo dessa série, e depois por dois termos, três termos, e assim por diante. Quantos termos dessa série você precisa usar antes de obter 3,14? 3,141? 3,1415? 3,14159?

**4.27 Triplas de Pitágoras.** Um triângulo retângulo pode ter lados que são valores inteiros. O conjunto de três valores inteiros para os lados de um triângulo retângulo é

chamado de tripla de Pitágoras. Esses três lados precisam satisfazer o relacionamento de que a soma dos quadrados de dois catetos é igual ao quadrado da hipotenusa. Ache todas as triplas de Pitágoras não superiores a 500 para cateto1, cateto2 e hipotenusa. Use um loop for triplamente aninhado que simplesmente teste todas as possibilidades. Este é um exemplo de computação por ‘força bruta’. Isso não é esteticamente atraente para muitas pessoas. Mas existem muitos motivos para essas técnicas serem importantes. Em primeiro lugar, com o poder da computação aumentando em um ritmo tão fenomenal, soluções que levariam anos, ou mesmo séculos, para serem produzidas com a tecnologia de alguns anos atrás agora podem ser produzidas em horas, minutos ou mesmo segundos. Os chips microprocessadores recentes podem processar um bilhão de instruções por segundo! Em segundo lugar, como você descobrirá em cursos de ciência da computação mais avançados, existem inúmeros problemas interessantes para os quais não existe uma técnica algorítmica conhecida além da simples força bruta. Investigamos muitos tipos de metodologias de solução de problemas neste livro. Consideraremos muitas técnicas de força bruta para diversos problemas interessantes.

**4.28** *Calculando o pagamento semanal.* Uma empresa paga a seus funcionários como gerentes (que recebem um salário semanal fixo), trabalhadores por hora (que recebem um salário fixo por hora até as 40 primeiras horas de trabalho e ‘hora e meia’ — ou seja, 1,5 vez o salário por hora — para horas extras trabalhadas), trabalhadores comissionados (que recebem R\$ 250,00 mais 5,7 por cento de suas vendas brutas semanais) ou trabalhadores por unidade (que recebem um valor fixo para cada um dos itens que eles produzem — cada trabalhador por unidade nessa empresa trabalha apenas em um tipo de item). Escreva um programa que calcule o pagamento semanal de cada empregado. Você não sabe o número de empregados com antecedência. Cada tipo de empregado tem seu próprio código de pagamento: gerentes têm código 1, trabalhadores por hora têm código 2, trabalhadores comissionados têm código 3 e trabalhadores por unidade têm código 4. Use um `switch` para calcular o pagamento de cada empregado com base no seu código de pagamento. Dentro do `switch`, peça ao usuário (ou seja, o funcionário administrativo da folha de pagamento) que informe os fatos apropriados que seu programa precisa para calcular o pagamento de cada empregado com base no código.

**4.29 Leis de De Morgan.** Neste capítulo, discutimos os operadores lógicos `&&`, `|` e `!`. Às vezes, as Leis de De Morgan podem tornar mais conveniente a indicação de uma expressão lógica. Essas leis afirmam que a expressão `!(condition1 && condition2)` é logicamente

equivalente à expressão  $!(condition1 \mid \mid !condition2)$ . Além disso, a expressão  $!(condition1 \mid \mid condition2)$  é logicamente equivalente à expressão  $!(condition1 \&\& !condition2)$ . Use as Leis de De Morgan para escrever expressões equivalentes para cada um dos seguintes itens, e depois escreva um programa para mostrar que a expressão original e a nova expressão em cada caso são equivalentes.

- a) `!( x < 5 ) && !( y >= 7 )`
- b) `!( a == b ) || !( g != 5 )`
- c) `!( ( x <= 8 ) && ( y > 4 ) )`
- d) `!( ( i > 4 ) || ( j <= 6 ) )`

**4.30** *Substituindo switch por if...else.* Reescreva o programa da Figura 4.7 substituindo a estrutura `switch` pela estrutura `if...else` aninhada; tenha o cuidado de tratar do caso `default` corretamente. Depois, reescreva essa nova versão substituindo a estrutura `if...else` aninhada por uma série de estruturas `if`; aqui, também tenha o cuidado de lidar com o caso `default` corretamente (isso é mais difícil do que na versão do `if...else` aninhado). Esse exercício demonstra que `switch` é uma conveniência, e que qualquer estrutura `switch` pode ser escrita apenas com instruções de seleção única.

**4.31 Programa de impressão de losango.** Escreva um programa que imprima a forma de losango a seguir. Você pode usar instruções `printf` que exibem um único asterisco (\*) ou um espaço em branco. Maximize o uso da repetição (com estruturas `for` aninhadas) e minimize o número de instruções `printf`.

```

 *

 *

```

**4.32 Programa de impressão de losango modificado.** Modifique o programa que você escreveu no Exercício 4.31 para ler um número ímpar no intervalo de 1 a 19, para especificar o número de linhas no losango. Seu programa deverá, então, exibir um losango com o tamanho apropriado.

**4.33 Equivalente de valores decimais em números romanos.** Escreva um programa que imprima uma tabela de todos os equivalentes em números romanos dos números decimais no intervalo de 1 a 100.



- 4.34** Descreva o processo que você usaria para substituir um loop `do...while` por um loop `while` equivalente. Que problema ocorre quando você tenta substituir um loop `while` por um loop `do...while` equivalente? Suponha que você tenha sido informado sobre a necessidade de remoção de um loop `while` e sua substituição por um `do...while`. Que instrução de controle adicional você precisaria usar, e como a usaria para garantir que o programa resultante se comportasse exatamente como o original?
- 4.35** Uma crítica ao comando `break` e ao comando `continue` é que eles não são estruturados. Na realidade, comandos `break` e comandos `continue` sempre podem ser substituídos por comandos estruturados, embora isso possa ser complicado. Descreva, em geral, como você removeria qualquer comando `break` de um loop em um programa e substituiria esse comando por algum equivalente estruturado. *[Dica: o comando `break` sai de um loop de dentro do corpo do loop. A outra maneira de sair é falhando no teste de continuação do loop. Considere o uso de um se-*

*gundo teste no teste de continuação do loop, que indique 'saída antecipada devido a uma condição de interrupção'.]* Use a técnica que você desenvolveu aqui para remover o comando `break` do programa da Figura 4.11.

- 4.36** O que o seguinte segmento de programa faz?

```
1 for (i = 1; i <= 5; i++) {
2 for (j = 1; j <= 3; j++) {
3 for (k = 1; k <= 4; k++)
4 printf("*");
5 printf("\n");
6 }
7 printf("\n");
8 }
```

- 4.37** Descreva, de modo geral, como você removeria qualquer comando `continue` de um loop em um programa e substituiria esse comando por algum equivalente estruturado. Use a técnica que você desenvolveu aqui para remover o comando `continue` do programa da Figura 4.12.

## ■ Fazendo a diferença

- 4.38** *Crescimento da população mundial.* A população mundial tem crescido consideravelmente no decorrer dos séculos. O crescimento continuado poderia, por fim, desafiar os limites do ar respirável, da água potável, do solo arável e de outros recursos limitados. Evidências indicam que, nos últimos anos, houve uma redução do crescimento, e que a população mundial poderia chegar a um pico em algum ano deste século, para depois começar a diminuir. Para este exercício, pesquise sobre questões de crescimento populacional on-line. *Não se esqueça de investigar diversos pontos de vista.* Encontre estimativas para a população mundial atual e sua taxa de crescimento (sua porcentagem de crescimento neste ano). Escreva um programa que calcule o crescimento da população mundial para os próximos 75 anos, *considerando a suposição simplificada de que a taxa de crescimento permanecerá constante.* Imprima os resultados em uma tabela. A primeira coluna deverá mostrar o ano, de 1 a 75. A segunda coluna deverá mostrar a população mundial antecipada ao final desse ano. A terceira coluna deverá mostrar o aumento numérico na população mundial que ocorreria nesse

ano. Usando os resultados obtidos, determine o ano em que a população dobraria o tamanho da população de hoje, se a taxa de crescimento desse ano persistisse.

- 4.39** *Alternativas de plano de imposto; a 'FairTax'.*

Existem muitas propostas para tornar a taxa mais justa. Verifique a iniciativa FairTax nos Estados Unidos em

[www.fairtax.org/site/PageServer?pagename=calculator](http://www.fairtax.org/site/PageServer?pagename=calculator)

Descubra como funciona a proposta FairTax. Uma sugestão é eliminar o imposto de renda e a maioria dos outros impostos em favor de uma taxa de consumo de 23 por cento sobre todos os produtos e serviços que você compra. Alguns oponentes da FairTax questionam o valor de 23 por cento e dizem que, devido ao modo como o imposto é calculado, seria mais exato dizer que a taxa é de 30 por cento — verifique isso cuidadosamente. Escreva um programa que leve o usuário a informar os gastos em diversas categorias (por exemplo, moradia, alimentação, vestuário, transporte, educação, saúde, férias), e depois imprima a FairTax estimada que essa pessoa pagaria.